SROTIX: A Production-Ready Retrieval-Augmented Generation Platform

Using FastAPI, React, and Vector Search

LITERATURE SURVEY

Student ID	Name
2520030359	Mohammad Faizaan
2520030267	Ishaan Kulkarni

1. Introduction

Large language models (LLMs) have enabled natural-language interfaces for information access, summarisation, and question answering. However, a model's internal parameters may not contain the latest or organisation-specific information, and generated answers can be difficult to verify. Retrieval-Augmented Generation (RAG) addresses this issue by retrieving relevant information from an external collection and using that evidence as context for generation. SROTIX applies this pattern to a full-stack document knowledge platform, with emphasis on traceable answers, operational reliability, and measurable evaluation.

2. Retrieval-Augmented Generation

Lewis et al. introduced a general RAG formulation that combines a neural retriever with a sequence-to-sequence generator, allowing generation to be conditioned on retrieved documents. Their work demonstrated the value of non-parametric knowledge for knowledge-intensive tasks [1]. For SROTIX, the central lesson is architectural: separate the document knowledge store from the generation model, retrieve evidence at query time, and preserve a path from the answer back to its supporting source. A practical application must also handle document ingestion, chunking, indexing, access control, and user-facing citations.

3. RAG Systems and Design Choices

Gao et al. survey the RAG landscape and discuss the evolution from a basic retrieve-then-generate pipeline toward more advanced approaches involving retrieval optimisation, reranking, query transformation, and iterative retrieval [2]. The survey highlights that RAG quality depends on several interacting stages rather than the language model alone. SROTIX therefore treats parsing, chunking, embeddings, retrieval, context construction, and answer generation as independently testable components. The initial version can use a straightforward dense-retrieval baseline, while later experiments can compare chunk sizes, overlap, top-k values, hybrid retrieval, and reranking.

4. Improving Reliability and Grounding

Self-RAG proposes a framework in which a model learns to retrieve, generate, and critique its own outputs using reflection signals, aiming to improve factuality and relevance [3]. Although SROTIX does not need to reproduce this research method, it reinforces the importance of making generation conditional on evidence and checking whether the response is supported. A practical first implementation can use simpler safeguards: instruct the model to answer only from retrieved context, return source identifiers and passages, and abstain when the evidence is insufficient. Such safeguards reduce risk but do not guarantee correctness, so citation support must be evaluated rather than assumed.

5. Evaluation of RAG Pipelines

RAG evaluation requires attention to both retrieval and generation. Retrieval metrics such as Recall@k measure whether relevant evidence appears among the top-k retrieved chunks. Answer-level assessment should examine correctness, relevance, and whether claims are supported by the cited context. Es et al. introduced RAGAS, a framework for reference-free evaluation of RAG pipelines using measures including faithfulness, answer relevance, and context relevance [4]. For SROTIX, a small curated test set with expected evidence and reference answers can provide a transparent baseline. Automated metrics may assist iteration, but a sample of outputs should also be reviewed by humans because evaluation scores can be imperfect proxies for factual quality.

6. Production-Oriented Engineering

Moving from a prototype to a dependable application introduces concerns that are not fully captured by retrieval benchmarks. Document parsing and embedding can be time-consuming, so ingestion should be handled through background jobs with observable states such as queued, processing, completed, and failed. Data models should preserve document ownership and knowledge-base membership, and API endpoints must enforce those boundaries on every read and write. Database migrations, structured logging, retries, upload validation, deletion semantics, automated tests, health checks, and reproducible deployment are also important. These engineering practices are proposed system requirements for SROTIX; they are not claims that the features have already been implemented.

7. Research Gap and Project Motivation

The foundational RAG paper establishes the retrieval-plus-generation approach, while later surveys and methods investigate ways to improve retrieval quality, grounding, and evaluation. However, an academic implementation project must integrate these ideas into a usable application with a complete lifecycle: uploading documents, indexing them, asking questions, inspecting evidence, managing conversations, and measuring system behaviour. SROTIX focuses on this integration. Its contribution is an engineering and evaluation-oriented implementation of a modular RAG platform, not a claim to invent a new RAG algorithm. The project will compare retrieval configurations and report observed limitations using a reproducible test set.

8. Proposed Methodology

The proposed workflow has two main paths. In ingestion, the backend validates the upload, extracts text, splits it into chunks, computes embeddings, and stores chunk text and metadata alongside vector representations. In question answering, the backend validates the user’s access to the selected knowledge base, embeds the query, retrieves the most relevant chunks, builds a bounded context, invokes a language model, and returns the answer with source references. The React frontend will expose knowledge-base and document management, ingestion status, chat history, and evidence inspection. Evaluation will record retrieval quality, citation support, answer quality, and latency. The initial baseline will be established before optional enhancements such as hybrid search or reranking are added.

9. Conclusion

The literature supports RAG as a practical method for connecting language generation with external evidence, while also showing that system quality depends on retrieval, grounding, and evaluation. SROTIX will apply these principles in a full-stack platform built with React and FastAPI, backed by PostgreSQL with vector search. Its evaluation plan and production-oriented requirements aim to make the system assessable, maintainable, and demonstrable. Final conclusions will be drawn from implementation and measured results rather than assumed in advance.

References

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS), 2020. Available: https://arxiv.org/abs/2005.11401

[2] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, 2023 (revised 2024). Available: https://arxiv.org/abs/2312.10997

[3] A. Asai et al., “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,” arXiv:2310.11511, 2023. Available: https://arxiv.org/abs/2310.11511

[4] S. Es et al., “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” arXiv:2309.15217, 2023. Available: https://arxiv.org/abs/2309.15217